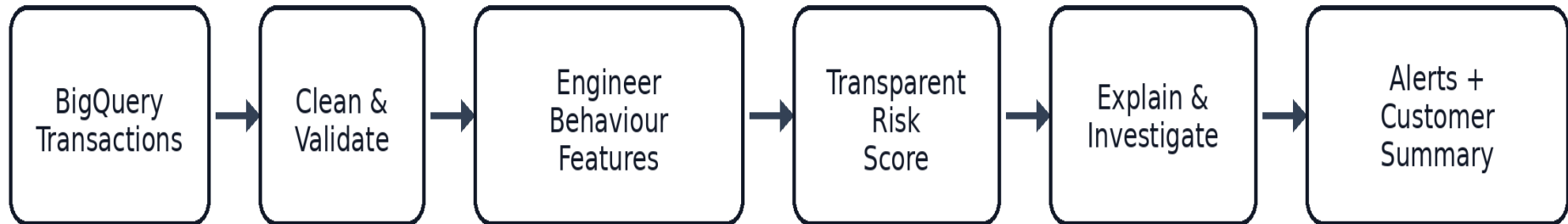


# AML Transaction Monitoring, Risk Scoring & Investigation Assistant

---

Simple-language concept guide based on the attached BigQuery and Python project



**Interview revision focus: data engineering, AML analytics, explainable scoring, validation and investigator support.**

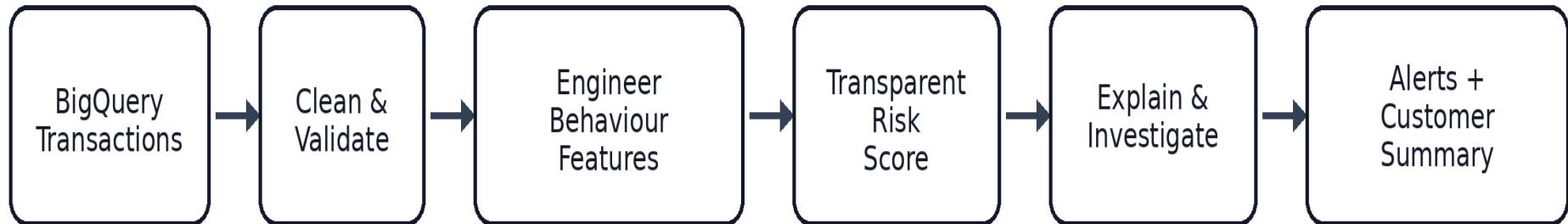


## Project at a glance

|                         |                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Business problem</b> | Millions of transactions cannot be reviewed manually. The project prioritises unusual transactions and accounts for human investigation. |
| <b>Data source</b>      | A BigQuery table containing sender, receiver, amount, currency, bank, payment format, time and an optional laundering label.             |
| <b>Core outputs</b>     | Transaction alert table, customer-level risk summary, plain-language risk reasons, recommended actions and draft case narratives.        |
| <b>Design principle</b> | A transparent score is used so investigators can see why an alert was created instead of receiving a black-box prediction.               |
| <b>Human control</b>    | The system recommends review. It does not make the final legal or compliance decision.                                                   |



## The Talent Grid | AML Transaction Monitoring Interview Guide





## 1. AML transaction monitoring

|                                  |                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | A system that watches financial transactions and highlights activity that may need investigation.                                          |
| <b>Abbreviation</b>              | AML = Anti-Money Laundering.                                                                                                               |
| <b>Why it matters</b>            | Criminal activity can be hidden inside ordinary-looking payments. Monitoring helps investigators focus on the riskiest activity first.     |
| <b>Project use</b>               | The notebook reads transaction data, creates behavioural indicators, calculates a risk score and writes alert tables for investigators.    |
| <b>What can go wrong</b>         | Too many alerts waste investigator time; too few alerts can miss suspicious activity. A score may also be misunderstood as proof of crime. |
| <b>How to validate / measure</b> | Track alert volume, alert rate, precision, recall, false positives, false negatives, investigator disposition and time saved.              |



## Professional Python implementation

```
# Count all transactions so the monitoring population is known.
total_transactions = len(df)

# Count transactions marked as alerts by the configured threshold.
total_alerts = int(df["is_alert"].sum())

# Calculate the percentage of transactions sent for review.
alert_rate = total_alerts / max(total_transactions, 1)

# Print the monitoring coverage for operational review.
print("Transactions:", total_transactions)
print("Alerts:", total_alerts)
print("Alert rate:", round(alert_rate, 4))
```



## 2. BigQuery data source and cloud analytics

|                                  |                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | BigQuery is a cloud data warehouse that stores and analyses very large tables using SQL.                                                   |
| <b>Abbreviation</b>              | SQL = Structured Query Language.                                                                                                           |
| <b>Why it matters</b>            | AML datasets can contain millions of rows. A cloud warehouse can scan, filter and aggregate them without loading everything into a laptop. |
| <b>Project use</b>               | The project connects to a Google Cloud project, reads the source table and later writes alert and customer-risk tables back to BigQuery.   |
| <b>What can go wrong</b>         | Wrong project IDs, permissions, table names or unlimited queries can cause failures, cost overruns or exposure of sensitive data.          |
| <b>How to validate / measure</b> | Confirm project and table IDs, inspect row counts and schema, use query limits during development, and review bytes processed.             |



## Professional Python implementation

```
# Import the official BigQuery client library.
from google.cloud import bigquery

# Create a client connected to the approved Google Cloud project.
client = bigquery.Client(project=PROJECT_ID)

# Build a fully qualified table reference to avoid querying the wrong table.
table_ref = f"{PROJECT_ID}.{DATASET_ID}.{SOURCE_TABLE_ID}"

# Retrieve table metadata before downloading data.
table = client.get_table(table_ref)

# Print the table size so the analyst can plan a safe query.
print("Rows reported by BigQuery:", table.num_rows)
```





### 3. Authentication and least-privilege access

|                                  |                                                                                                                 |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Authentication proves who the user is. Authorisation decides what that user is allowed to do.                   |
| <b>Abbreviation</b>              | IAM = Identity and Access Management.                                                                           |
| <b>Why it matters</b>            | Transaction data is sensitive. Only approved users and service accounts should read or write it.                |
| <b>Project use</b>               | The notebook authenticates the Colab user and creates a BigQuery client for the configured project.             |
| <b>What can go wrong</b>         | Overly broad access can expose data or allow accidental table deletion. Missing access blocks the pipeline.     |
| <b>How to validate / measure</b> | Test read and write permissions separately, review IAM roles, inspect audit logs and remove unused credentials. |

#### Professional Python implementation

```
# Import Colab authentication support.
from google.colab import auth

# Ask the current user to authenticate with Google.
auth.authenticate_user()

# Create a project-scoped BigQuery client after authentication succeeds.
client = bigquery.Client(project=PROJECT_ID)

# Display the connected project to prevent silent use of the wrong environment.
print("Connected project:", client.project)
```



## 4. Schema inspection and data contracts

|                                  |                                                                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | A schema describes each column name and data type. A data contract defines what the pipeline expects.                                     |
| <b>Abbreviation</b>              | ID = Identifier.                                                                                                                          |
| <b>Why it matters</b>            | Code may silently produce wrong results when a date arrives as text or an amount arrives in the wrong column.                             |
| <b>Project use</b>               | The notebook prints the BigQuery schema and searches for expected timestamp, bank and account columns.                                    |
| <b>What can go wrong</b>         | Renamed or duplicated account columns can mix sender and receiver information. Wrong types can break calculations.                        |
| <b>How to validate / measure</b> | Compare the actual schema with required fields, test data types, count missing required columns and stop the job when the contract fails. |



## Professional Python implementation

```
# Define the columns required for safe transaction analysis.
required_columns = {
    "transaction_timestamp",
    "from_bank",
    "from_account",
    "to_bank",
    "to_account",
    "amount_paid",
    "payment_currency",
}

# Find required columns that are absent from the dataframe.
missing_columns = sorted(required_columns - set(df.columns))

# Stop immediately when the data contract is broken.
if missing_columns:
    raise ValueError(f"Missing required columns: {missing_columns}")
```



## 5. Column-name standardisation

|                                  |                                                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Standardisation converts inconsistent names into one predictable format, such as lowercase snake_case.                         |
| <b>Abbreviation</b>              | snake_case = words separated with underscores.                                                                                 |
| <b>Why it matters</b>            | `From Bank`, `from-bank` and `FROM_BANK` should not be treated as different ideas.                                             |
| <b>Project use</b>               | The notebook removes special characters, replaces spaces with underscores and converts names to lowercase.                     |
| <b>What can go wrong</b>         | Two original columns can become the same standardised name, or code can select the wrong automatically renamed account column. |
| <b>How to validate / measure</b> | Check for duplicate names after standardisation and compare mapped sender/receiver columns with source samples.                |



## Professional Python implementation

```
# Import regular-expression support for controlled text replacement.
import re

# Define one reusable function for consistent column naming.
def normalise_column_name(name: str) -> str:
    # Convert any input label to text and remove outer spaces.
    name = str(name).strip()
    # Replace groups of non-alphanumeric characters with one underscore.
    name = re.sub(r"[^0-9a-zA-Z]+", "_", name)
    # Collapse repeated underscores and remove underscores at both ends.
    name = re.sub(r"_+", "_", name).strip("_")
    # Return lowercase snake_case for predictable downstream references.
    return name.lower()

# Apply the function to every dataframe column.
df.columns = [normalise_column_name(column) for column in df.columns]

# Detect collisions caused by standardisation.
assert not df.columns.duplicated().any(), "Duplicate column names detected."
```



## 6. Data cleaning and validation

|                                  |                                                                                                                              |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Cleaning converts raw values into usable formats. Validation checks whether those values make sense.                         |
| <b>Abbreviation</b>              | NaN = Not a Number; UTC = Coordinated Universal Time.                                                                        |
| <b>Why it matters</b>            | Risk features are only reliable when timestamps, numbers, currencies and account IDs are correctly prepared.                 |
| <b>Project use</b>               | The notebook converts dates and amounts, fills missing text safely, creates entity keys and checks transaction integrity.    |
| <b>What can go wrong</b>         | Invalid dates may become missing, negative amounts may distort risk, and blank account IDs may merge unrelated customers.    |
| <b>How to validate / measure</b> | Measure missingness, invalid timestamps, negative or zero amounts, duplicate transaction IDs and impossible currency values. |



## Professional Python implementation

```
# Convert the transaction time to a timezone-aware datetime.
df["transaction_timestamp"] = pd.to_datetime(
    df["transaction_timestamp"],
    errors="coerce",
    utc=True,
)

# Convert payment amount to numeric and turn invalid text into missing values.
df["amount_paid"] = pd.to_numeric(df["amount_paid"], errors="coerce")

# Remove rows that cannot support time-based or amount-based analysis.
df = df.dropna(subset=["transaction_timestamp", "amount_paid"]).copy()

# Keep only economically meaningful positive payment amounts.
df = df.loc[df["amount_paid"] > 0].copy()

# Confirm the cleaned data still contains records.
assert len(df) > 0, "No valid transactions remain after cleaning."
```



## 7. Data-quality profiling

|                                  |                                                                                                                                  |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Profiling creates a quick health report for every column.                                                                        |
| <b>Abbreviation</b>              | DQ = Data Quality.                                                                                                               |
| <b>Why it matters</b>            | Before modelling risk, the analyst must understand missing values, distinct values and data types.                               |
| <b>Project use</b>               | The notebook builds a table with column name, type, missing count, missing percentage and unique count.                          |
| <b>What can go wrong</b>         | A pipeline may appear successful while key columns are almost empty or contain one repeated value.                               |
| <b>How to validate / measure</b> | Use missing percentage, uniqueness, duplicate counts, range checks, category frequency and trend comparisons with previous runs. |





## Professional Python implementation

```
# Build one row of quality statistics for every dataframe column.
quality_summary = pd.DataFrame({
    # Store the column names being assessed.
    "column": df.columns,
    # Record the current pandas data type.
    "dtype": [str(df[column].dtype) for column in df.columns],
    # Count missing values in each column.
    "missing_count": [int(df[column].isna().sum()) for column in df.columns],
    # Calculate the missing-value percentage for easier comparison.
    "missing_pct": [
        round(float(df[column].isna().mean() * 100), 2)
        for column in df.columns
    ],
    # Count distinct values, including missing as a possible state.
    "unique_count": [
        int(df[column].nunique(dropna=False))
        for column in df.columns
    ],
})

# Sort the report so the most incomplete columns appear first.
quality_summary = quality_summary.sort_values(
    "missing_pct",
    ascending=False,
)
```



## 8. Entity keys and transaction identifiers

|                                  |                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | An entity key combines fields to represent one sender or receiver consistently.                                     |
| <b>Abbreviation</b>              | ID = Identifier.                                                                                                    |
| <b>Why it matters</b>            | The same account number can exist at different banks, so account alone may not be unique.                           |
| <b>Project use</b>               | The notebook combines bank and account to create sender_entity and receiver_entity, then creates a transaction_id.  |
| <b>What can go wrong</b>         | Weak keys can merge different customers, split one customer into many identities or double-count transactions.      |
| <b>How to validate / measure</b> | Test uniqueness, inspect collisions, compare with master customer data and reconcile counts with the source system. |



## Professional Python implementation

```
# Build a sender key from both bank and account to reduce collisions.
df["sender_entity"] = (
    df["from_bank"].astype(str).str.strip()
    + "::"
    + df["from_account"].astype(str).str.strip()
)

# Build the equivalent receiver key.
df["receiver_entity"] = (
    df["to_bank"].astype(str).str.strip()
    + "::"
    + df["to_account"].astype(str).str.strip()
)

# Create a deterministic transaction ID when no trusted source ID exists.
df["transaction_id"] = (
    df.index.astype(str)
    + "-"
    + df["transaction_timestamp"].astype(str)
)

# Verify that every generated transaction ID is unique.
assert df["transaction_id"].is_unique
```



## 9. Behavioural feature engineering

|                                  |                                                                                                                                                                     |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Feature engineering turns raw transactions into measurements that describe behaviour.                                                                               |
| <b>Abbreviation</b>              | Feature = a measurable input used by a rule or model.                                                                                                               |
| <b>Why it matters</b>            | A single amount may look normal, but repeated transfers, many receivers or unusual timing may reveal risk.                                                          |
| <b>Project use</b>               | The notebook creates daily counts, daily amounts, unique receivers, receiver sender counts, time gaps and deviation measures.                                       |
| <b>What can go wrong</b>         | Data leakage, incorrect grouping, unsorted timestamps or future information can make performance look better than reality.                                          |
| <b>How to validate / measure</b> | Recalculate features on a small hand-checked sample, test time ordering, compare group totals and ensure each feature uses only information available at that time. |



## Professional Python implementation

```
# Sort transactions before calculating time-dependent behaviour.
df = df.sort_values(
    ["sender_entity", "transaction_timestamp"]
).copy()

# Extract the calendar day used for same-day aggregations.
df["transaction_date"] = df["transaction_timestamp"].dt.date

# Count how many transactions each sender made on each day.
df["sender_daily_txn_count"] = df.groupby(
    ["sender_entity", "transaction_date"]
)["transaction_id"].transform("count")

# Sum the sender's total payment amount for that day.
df["sender_daily_amount"] = df.groupby(
    ["sender_entity", "transaction_date"]
)["transaction_amount"].transform("sum")

# Count how many different receivers the sender paid that day.
df["sender_daily_unique_receivers"] = df.groupby(
    ["sender_entity", "transaction_date"]
)["receiver_entity"].transform("nunique")
```



## 10. Velocity and rapid-repeat transfers

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Velocity measures how quickly and how often money moves.                                                                 |
| <b>Abbreviation</b>              | Txn = Transaction.                                                                                                       |
| <b>Why it matters</b>            | Many transactions in a short period can indicate automation, layering or deliberate splitting.                           |
| <b>Project use</b>               | The notebook compares each sender transaction with the previous timestamp and marks another transfer within ten minutes. |
| <b>What can go wrong</b>         | Unsorted data, mixed time zones or duplicated timestamps can create false rapid-transfer flags.                          |
| <b>How to validate / measure</b> | Inspect time gaps, compare flagged sequences with raw timestamps and measure alert precision for different time windows. |



## Professional Python implementation

```
# Find the previous transaction time for each sender.
df["previous_sender_timestamp"] = df.groupby(
    "sender_entity"
)["transaction_timestamp"].shift(1)

# Calculate elapsed minutes since the sender's previous transaction.
df["minutes_since_previous_sender_txn"] = (
    df["transaction_timestamp"]
    - df["previous_sender_timestamp"]
).dt.total_seconds() / 60.0

# Flag a repeated transfer occurring within ten minutes.
df["rapid_repeat_transfer"] = (
    df["minutes_since_previous_sender_txn"]
    .between(0, 10, inclusive="both")
    .fillna(False)
    .astype(int)
)
```



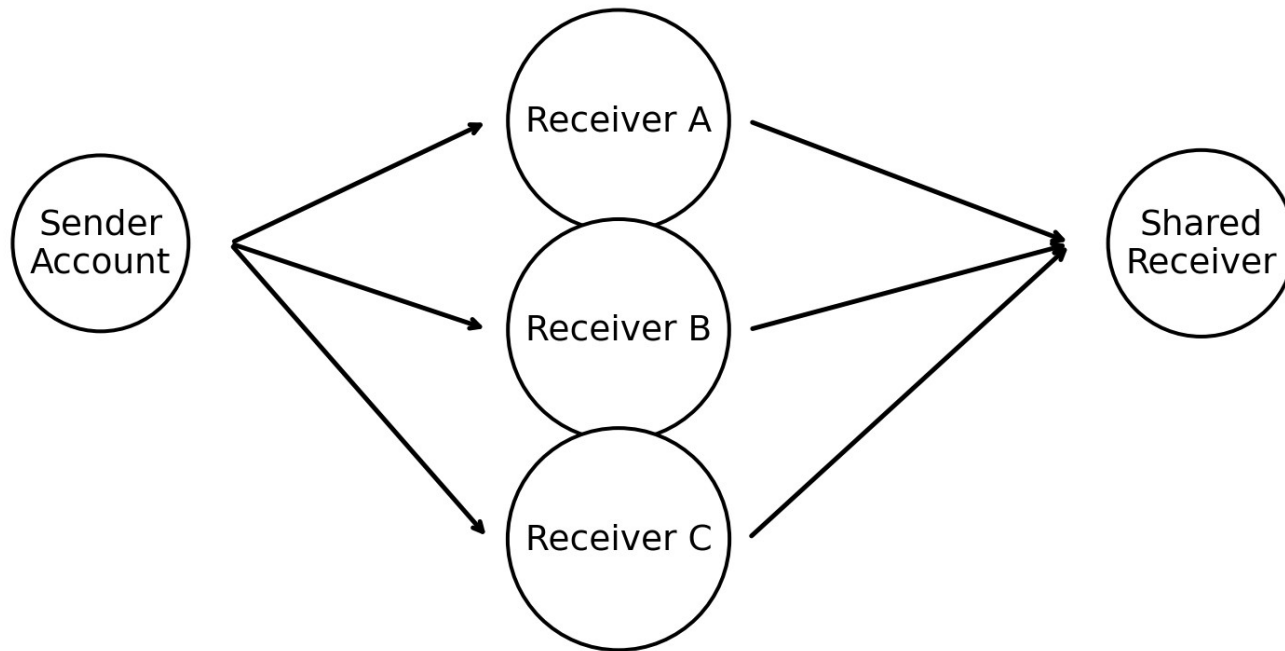
## 11. Network and counterparty behaviour

|                                  |                                                                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Network analysis studies who sends money to whom and how many connections each party has.                               |
| <b>Abbreviation</b>              | Fan-out = one sender pays many receivers; fan-in = one receiver collects from many senders.                             |
| <b>Why it matters</b>            | Money-laundering behaviour may be visible in the relationship pattern rather than in one transaction.                   |
| <b>Project use</b>               | The notebook counts a sender's unique receivers and a receiver's unique senders to capture fan-out and fan-in.          |
| <b>What can go wrong</b>         | Shared business accounts, payment processors and payroll accounts may look suspicious without customer context.         |
| <b>How to validate / measure</b> | Compare with customer type, peer groups and known business purpose; inspect network diagrams and investigator outcomes. |





## The Talent Grid | AML Transaction Monitoring Interview Guide



Network features reveal fan-out, fan-in and linked activity patterns.



## Professional Python implementation

```
# Count the number of distinct senders linked to each receiver.
df["receiver_unique_senders"] = df.groupby(
    "receiver_entity"
)["sender_entity"].transform("nunique")

# Flag receivers collecting from many different senders.
df["high_fan_in_receiver"] = (
    df["receiver_unique_senders"] >= 5
).astype(int)

# Flag senders paying many different receivers in one day.
df["high_fan_out_sender"] = (
    df["sender_daily_unique_receivers"] >= 4
).astype(int)
```



## 12. Statistical deviation and z-score

|                                  |                                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | A deviation feature asks whether a transaction is unusual for that sender.                                                                 |
| <b>Abbreviation</b>              | Z-score = number of standard deviations a value is above or below the mean.                                                                |
| <b>Why it matters</b>            | A ₹5 lakh transfer may be normal for one business and highly unusual for another customer.                                                 |
| <b>Project use</b>               | The notebook compares each transaction amount with the sender's historical mean and standard deviation.                                    |
| <b>What can go wrong</b>         | New customers have little history, outliers distort the mean, and zero standard deviation causes division problems.                        |
| <b>How to validate / measure</b> | Check distribution by customer, use minimum-history rules, compare robust alternatives such as median and MAD, and inspect extreme values. |



## Professional Python implementation

```
# Calculate each sender's average historical transaction amount.
sender_mean = df.groupby("sender_entity")[
    "transaction_amount"
].transform("mean")

# Calculate each sender's transaction-amount standard deviation.
sender_std = df.groupby("sender_entity")[
    "transaction_amount"
].transform("std")

# Avoid division by zero for senders with no variation.
safe_std = sender_std.replace(0, np.nan)

# Measure how many standard deviations each transaction is from the sender mean.
df["amount_zscore"] = (
    (df["transaction_amount"] - sender_mean) / safe_std
).fillna(0.0)

# Flag strongly unusual positive deviations.
df["high_amount_deviation"] = (
    df["amount_zscore"] >= 3.0
).astype(int)
```



## 13. Pattern indicators: structuring, round amounts and unusual hours

|                                  |                                                                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Pattern indicators are understandable rules that point to known suspicious behaviours.                                      |
| <b>Abbreviation</b>              | Structuring = splitting money into several smaller transfers to avoid attention.                                            |
| <b>Why it matters</b>            | Transparent rules make it easier for investigators to understand the alert.                                                 |
| <b>Project use</b>               | The notebook flags multiple same-day transfers, large round amounts, cash-like formats, self-transfers and unusual hours.   |
| <b>What can go wrong</b>         | Fixed thresholds may be too strict for one market and too loose for another. Legitimate customer habits can trigger alerts. |
| <b>How to validate / measure</b> | Back-test each rule, review hit rates by customer segment, tune thresholds and require investigator feedback.               |



## Professional Python implementation

```
# Flag transactions made during low-activity overnight hours.
df["is_unusual_hour"] = (
    df["transaction_timestamp"].dt.hour.isin([0, 1, 2, 3, 4, 5])
).astype(int)

# Flag large values that are exact multiples of 1,000.
df["is_round_amount"] = (
    (df["transaction_amount"] >= 10000)
    & (df["transaction_amount"] % 1000 == 0)
).astype(int)

# Flag possible structuring when a sender makes several same-day payments.
df["potential_structuring"] = (
    (df["sender_daily_txn_count"] >= 3)
    & (df["sender_daily_amount"] >= 10000)
).astype(int)

# Flag transactions where sender and receiver are the same entity.
df["self_transfer"] = (
    df["sender_entity"] == df["receiver_entity"]
).astype(int)
```

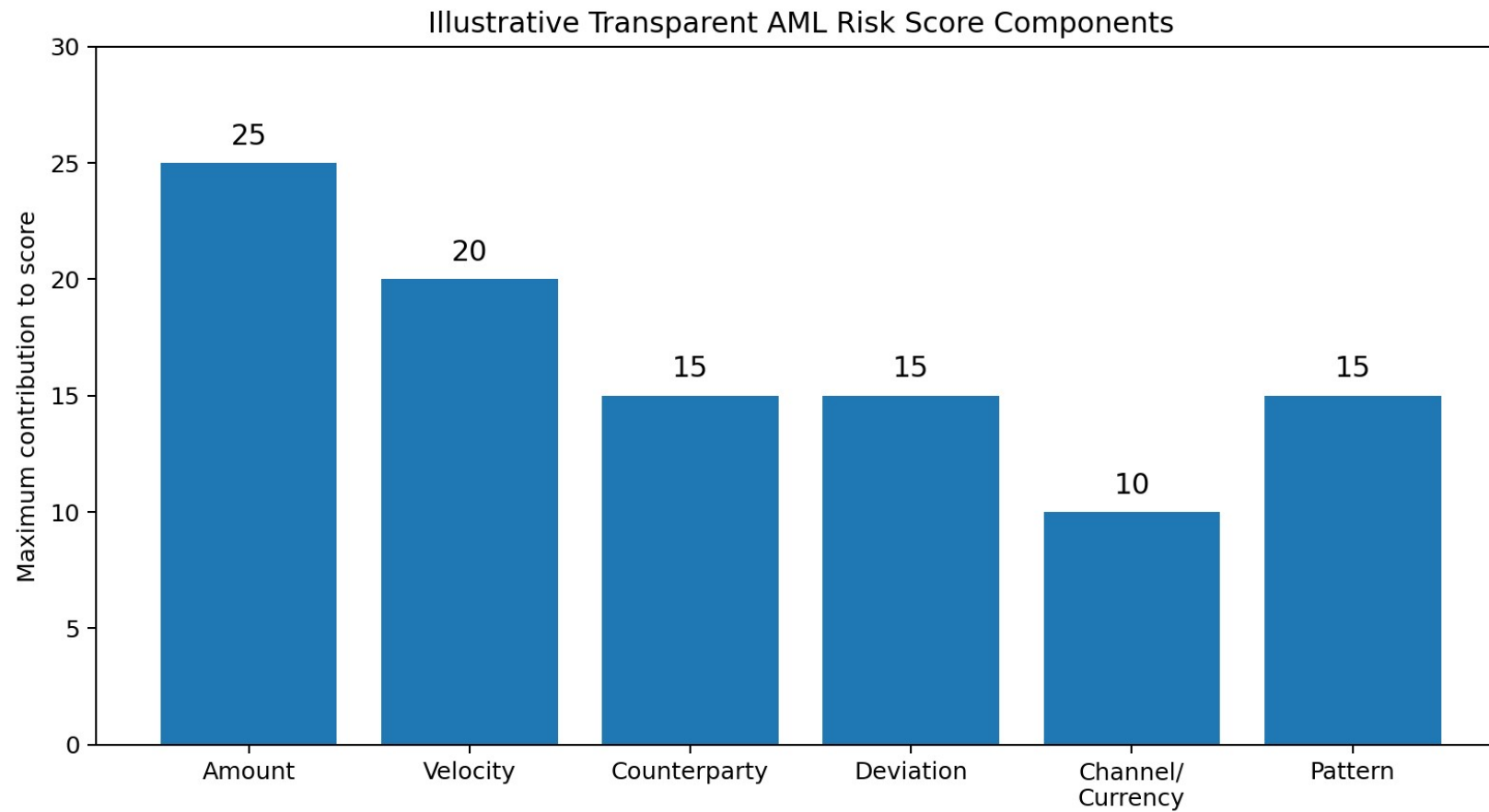


## 14. Transparent weighted risk score

|                                  |                                                                                                                                      |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | A weighted score adds several risk components, with more important signals contributing more points.                                 |
| <b>Abbreviation</b>              | Score range in this project: 0 to 100.                                                                                               |
| <b>Why it matters</b>            | Investigators can see which behaviours increased risk instead of trusting an unexplained model output.                               |
| <b>Project use</b>               | Amount, velocity, counterparty, deviation, channel/currency and pattern components are combined and clipped to 100.                  |
| <b>What can go wrong</b>         | Poor weights can over-alert one behaviour, hide another or create a false sense of mathematical certainty.                           |
| <b>How to validate / measure</b> | Compare score bands with known labels and investigator outcomes, perform sensitivity tests and monitor score distribution over time. |



## The Talent Grid | AML Transaction Monitoring Interview Guide







## Professional Python implementation

```
# Convert transaction amount into a relative 0-to-1 rank.
amount_rank = df["transaction_amount"].rank(
    pct=True,
    method="average",
)

# Give high-value transactions up to 25 risk points.
df["amount_risk_component"] = 25 * amount_rank

# Give transaction velocity up to 20 risk points.
df["velocity_risk_component"] = (
    10 * (df["sender_daily_txn_count"] >= 3).astype(int)
    + 10 * df["rapid_repeat_transfer"]
)

# Add the transparent components and cap the final score at 100.
df["risk_score"] = (
    df["amount_risk_component"]
    + df["velocity_risk_component"]
    + df["counterparty_risk_component"]
    + df["behaviour_deviation_component"]
    + df["channel_currency_component"]
    + df["pattern_component"]
).clip(lower=0, upper=100)
```



## 15. Thresholds, alerts and risk levels

|                                  |                                                                                                                         |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | A threshold is the score boundary used to create an alert. Risk levels group scores into understandable bands.          |
| <b>Abbreviation</b>              | LOW, MEDIUM, HIGH and CRITICAL are operational severity labels.                                                         |
| <b>Why it matters</b>            | The threshold controls workload and detection. A lower threshold creates more alerts; a higher threshold creates fewer. |
| <b>Project use</b>               | The notebook uses a score of 60 or more as an alert and categorises the score into four levels.                         |
| <b>What can go wrong</b>         | A threshold copied from development may overload investigators in production or miss important activity.                |
| <b>How to validate / measure</b> | Review precision-recall trade-offs, alert capacity, false-negative cost, case-conversion rate and threshold stability.  |



## Professional Python implementation

```
# Set the operational score boundary for creating an alert.
ALERT_THRESHOLD = 60.0

# Mark transactions meeting or exceeding the threshold.
df["is_alert"] = (
    df["risk_score"] >= ALERT_THRESHOLD
).astype(int)

# Convert numeric scores into human-readable risk bands.
df["risk_level"] = pd.cut(
    df["risk_score"],
    bins=[-np.inf, 30, 60, 80, np.inf],
    labels=["LOW", "MEDIUM", "HIGH", "CRITICAL"],
    right=False,
).astype(str)
```



## 16. Explainability and risk reasons

|                                  |                                                                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Explainability tells the investigator which facts caused the score.                                                                     |
| <b>Abbreviation</b>              | XAI = Explainable Artificial Intelligence.                                                                                              |
| <b>Why it matters</b>            | Compliance teams need defensible reasons, not just a number.                                                                            |
| <b>Project use</b>               | The notebook builds up to six plain-language reasons, such as large value, many receivers, rapid repetition or cross-currency transfer. |
| <b>What can go wrong</b>         | Reasons may be technically true but misleading without customer context, or may not exactly match the score logic.                      |
| <b>How to validate / measure</b> | Unit-test every reason, reconcile reason text with component values and ask investigators whether explanations are actionable.          |



## Professional Python implementation

```
# Start an empty explanation list for one transaction.
reasons = []

# Add a reason when the transaction amount is unusually high.
if row["amount_zscore"] >= 3:
    reasons.append(
        "Transaction amount is far above the sender's normal level."
    )

# Add a reason when the sender paid many receivers on the same day.
if row["sender_daily_unique_receivers"] >= 4:
    reasons.append(
        "Sender paid several different receivers on the same day."
    )

# Provide a safe fallback when risk comes from several moderate signals.
if not reasons:
    reasons.append(
        "Risk arises from the combined transaction and behavioural profile."
    )
```



## 17. Investigation assistant and human-in-the-loop review

|                                  |                                                                                                                            |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | The assistant turns analytics into practical review steps and a draft narrative.                                           |
| <b>Abbreviation</b>              | HITL = Human in the Loop.                                                                                                  |
| <b>Why it matters</b>            | Investigators need context, recommended checks and consistent documentation, but a person must make the final decision.    |
| <b>Project use</b>               | The notebook recommends document checks, customer-behaviour comparison, network review and escalation for high-risk cases. |
| <b>What can go wrong</b>         | Generated narratives can sound certain, contain incorrect context or encourage automatic decisions.                        |
| <b>How to validate / measure</b> | Require human approval, sample case quality, compare narrative facts with source columns and record investigator edits.    |



## Professional Python implementation

```
# Define baseline review actions that apply to every alert.
actions = [
    "Verify the transaction purpose and supporting documentation.",
    "Compare activity with the customer's expected account behaviour.",
]

# Add a targeted action when possible structuring is present.
if row["potential_structuring"] == 1:
    actions.append(
        "Review same-day transfers for linked or deliberately split payments."
    )

# Escalate severe cases only after contextual evidence is reviewed.
if row["risk_level"] in {"HIGH", "CRITICAL"}:
    actions.append(
        "Escalate for enhanced review if evidence does not explain the activity."
    )
```



## 18. Transaction alert table

|                                  |                                                                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | An alert table contains only transactions selected for investigation, together with evidence and actions.                   |
| <b>Abbreviation</b>              | Case queue = the ordered list of alerts awaiting review.                                                                    |
| <b>Why it matters</b>            | Investigators need a compact, prioritised dataset instead of the full transaction population.                               |
| <b>Project use</b>               | The notebook selects alert columns, filters <code>is_alert = 1</code> and sorts by risk score from highest to lowest.       |
| <b>What can go wrong</b>         | Missing evidence columns prevent investigation; duplicated alerts create wasted work; sensitive columns may be overexposed. |
| <b>How to validate / measure</b> | Reconcile alert counts, verify sort order, test required fields, check duplicates and confirm access controls.              |





## Professional Python implementation

```
# Select only records that crossed the alert threshold.
alerts = df.loc[
    df["is_alert"] == 1,
    alert_columns,
].copy()

# Sort the queue so the highest-risk cases are reviewed first.
alerts = alerts.sort_values(
    "risk_score",
    ascending=False,
).reset_index(drop=True)

# Verify that every row in the queue is truly an alert.
assert alerts["is_alert"].eq(1).all()
```



## 19. Customer-level risk aggregation

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Aggregation combines many transactions into one summary row for each sender account.                                     |
| <b>Abbreviation</b>              | KYC = Know Your Customer.                                                                                                |
| <b>Why it matters</b>            | A customer may look risky because of repeated behaviour even when no single transaction appears extreme.                 |
| <b>Project use</b>               | The notebook calculates totals, averages, maximums, unique receivers, alert counts, alert rate and maximum risk score.   |
| <b>What can go wrong</b>         | Aggregation can hide timing, mix unrelated entities or label a customer using one erroneous transaction.                 |
| <b>How to validate / measure</b> | Reconcile account totals, drill down from customer to transactions, test key quality and compare with KYC/customer type. |



## Professional Python implementation

```
# Group transaction-level data into one row per sender entity.
customer_risk = df.groupby("sender_entity").agg(
    # Count all transactions initiated by the sender.
    total_transactions=("transaction_id", "count"),
    # Sum the sender's transaction amount.
    total_amount=("transaction_amount", "sum"),
    # Count how many alerts were generated for the sender.
    alert_count=("is_alert", "sum"),
    # Keep the highest transaction-level risk score.
    maximum_risk_score=("risk_score", "max"),
    # Count distinct counterparties paid by the sender.
    unique_receivers=("receiver_entity", "nunique"),
).reset_index()

# Calculate the proportion of the sender's transactions that became alerts.
customer_risk["alert_rate"] = (
    customer_risk["alert_count"]
    / customer_risk["total_transactions"].replace(0, np.nan)
).fillna(0.0)
```



## 20. Model validation with labelled data

|                                  |                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Validation compares alerts with known outcomes to understand how well the system detects labelled laundering cases.                           |
| <b>Abbreviation</b>              | TP = True Positive; FP = False Positive; FN = False Negative; TN = True Negative.                                                             |
| <b>Why it matters</b>            | A score is not trustworthy until its behaviour is measured on relevant data.                                                                  |
| <b>Project use</b>               | When the IBM laundering label is available, the notebook calculates a confusion matrix, classification report, ROC-AUC and average precision. |
| <b>What can go wrong</b>         | Labels may be incomplete, simulated or not representative of current production behaviour. Accuracy can be misleading for rare crime events.  |
| <b>How to validate / measure</b> | Prioritise precision, recall, F1, PR-AUC, alert workload, stability and investigator outcomes; validate on separate time periods.             |



## Professional Python implementation

```
# Store the known laundering label as the validation target.
y_true = df["is_laundering"].astype(int)

# Store the pipeline's binary alert decision.
y_pred = df["is_alert"].astype(int)

# Convert the 0-to-100 score into a 0-to-1 ranking value.
y_score = df["risk_score"] / 100.0

# Print the confusion matrix to inspect TP, FP, FN and TN counts.
print(confusion_matrix(y_true, y_pred))

# Measure ranking quality for the rare positive class.
print("Average precision:", average_precision_score(y_true, y_score))
```



## 21. Precision, recall, F1 and threshold tuning

|                                  |                                                                                                                           |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | Precision asks how many alerts were truly positive. Recall asks how many true positives were found. F1 balances both.     |
| <b>Abbreviation</b>              | F1 = harmonic mean of precision and recall; PR = Precision-Recall.                                                        |
| <b>Why it matters</b>            | AML teams must balance missed risk against investigator workload.                                                         |
| <b>Project use</b>               | The notebook calculates precision and recall at many thresholds, then identifies the threshold with the best F1 score.    |
| <b>What can go wrong</b>         | The mathematically best F1 threshold may not match regulatory risk appetite or available case capacity.                   |
| <b>How to validate / measure</b> | Review the full precision-recall curve, expected cases per day, cost of missed cases and performance by customer segment. |



## Professional Python implementation

```
# Calculate precision and recall across possible score thresholds.
precision, recall, thresholds = precision_recall_curve(
    y_true,
    y_score,
)

# Align threshold values with the precision and recall arrays.
threshold_table = pd.DataFrame({
    "threshold": np.r_[thresholds, np.nan],
    "precision": precision,
    "recall": recall,
})

# Calculate F1 while safely handling a zero denominator.
threshold_table["f1"] = (
    2
    * threshold_table["precision"]
    * threshold_table["recall"]
    / (
        threshold_table["precision"]
        + threshold_table["recall"]
    ).replace(0, np.nan)
)

# Select the row with the highest measured F1 value.
best_threshold_row = threshold_table.loc[
    threshold_table["f1"].idxmax()
]
```



## 22. Writing governed outputs back to BigQuery

|                                  |                                                                                                            |
|----------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | The pipeline saves approved results as managed cloud tables for dashboards and investigator queries.       |
| <b>Abbreviation</b>              | WRITE_TRUNCATE = replace the destination table with the new run.                                           |
| <b>Why it matters</b>            | Operational users need stable outputs that can be queried independently of the notebook.                   |
| <b>Project use</b>               | The notebook uploads transaction alerts and customer summaries to two BigQuery tables.                     |
| <b>What can go wrong</b>         | WRITE_TRUNCATE can erase history, schema drift can fail uploads and repeated runs may remove auditability. |
| <b>How to validate / measure</b> | Check destination row counts, schema, load-job status, run timestamps, lineage and retention strategy.     |





## Professional Python implementation

```
# Configure the load to replace the previous table contents.
job_config = bigquery.LoadJobConfig(
    write_disposition="WRITE_TRUNCATE",
)

# Start the alert-table upload to BigQuery.
alert_load_job = client.load_table_from_dataframe(
    alerts_to_upload,
    OUTPUT_ALERT_TABLE,
    job_config=job_config,
)

# Wait for the job to finish so failures are not ignored.
alert_load_job.result()

# Confirm the destination table now contains the expected number of rows.
uploaded_table = client.get_table(OUTPUT_ALERT_TABLE)
assert uploaded_table.num_rows == len(alerts_to_upload)
```



## 23. Investigator SQL queries

|                                  |                                                                                                                   |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Simple meaning</b>            | SQL queries let investigators retrieve the highest-risk alerts and repeated high-risk accounts.                   |
| <b>Abbreviation</b>              | ORDER BY = sort results; LIMIT = return only a selected number of rows.                                           |
| <b>Why it matters</b>            | Operational teams need fast, repeatable ways to find cases requiring attention.                                   |
| <b>Project use</b>               | The notebook queries the alert table by descending risk score and filters customer summaries to HIGH or CRITICAL. |
| <b>What can go wrong</b>         | Queries without filters can scan too much data, expose sensitive fields or produce inconsistent case lists.       |
| <b>How to validate / measure</b> | Use approved views, parameterised filters, row-level security, query-cost monitoring and result reconciliation.   |



## Professional Python implementation

```
# Define a query that returns the highest-risk transaction alerts.
top_alerts_sql = f'''
SELECT
    transaction_id,
    transaction_timestamp,
    from_bank,
    from_account,
    to_bank,
    to_account,
    transaction_amount,
    risk_score,
    risk_level,
    risk_reasons
FROM `{OUTPUT_ALERT_TABLE}`
ORDER BY risk_score DESC
LIMIT 100
'''

# Execute the approved query and load the result into pandas.
top_alerts = client.query(top_alerts_sql).to_dataframe()
```



## Production readiness checklist

|                          |                                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>Data governance</b>   | Encrypt sensitive data, apply least privilege, mask account details where possible and retain audit logs. |
| <b>Rule governance</b>   | Document every rule, owner, rationale, version, threshold and approval date.                              |
| <b>Model risk</b>        | Validate on separate periods and customer segments; monitor drift and false-negative exposure.            |
| <b>Operations</b>        | Set alert-capacity limits, prioritisation rules, SLAs and escalation paths.                               |
| <b>Human oversight</b>   | Require investigators to approve or reject alerts and record reasons.                                     |
| <b>Auditability</b>      | Store run ID, code version, data date, score components, explanations and final disposition.              |
| <b>Monitoring</b>        | Track data quality, alert rate, precision, recall, average case time and changes in score distribution.   |
| <b>Change management</b> | Test updates in a non-production environment and use controlled deployment.                               |



## Interview revision: 12 questions with strong answer points

| Interview question                                                 | Strong answer points                                                                                                                            |
|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Why use a transparent score instead of only a black-box model?     | AML decisions require explainability, auditability and investigator trust. A transparent score also makes threshold and rule governance easier. |
| What is the difference between transaction risk and customer risk? | Transaction risk evaluates one payment. Customer risk aggregates behaviour across many payments and counterparties.                             |
| Why sort by sender and timestamp before velocity features?         | Time gaps and previous-transaction features are only correct when events are in chronological order for each sender.                            |
| Why can accuracy be misleading?                                    | Laundering labels are rare, so a model can achieve high accuracy by predicting almost everything as normal.                                     |
| What does precision mean operationally?                            | Of all alerts sent to investigators, precision is the proportion that match the positive label.                                                 |
| What does recall mean operationally?                               | Of all labelled laundering transactions, recall is the proportion detected by the system.                                                       |
| Why use BigQuery?                                                  | It supports scalable SQL, governed cloud storage, large-table aggregation and shared operational outputs.                                       |
| What is data leakage?                                              | Using future or outcome information that would not be available at the time of scoring.                                                         |
| Why combine bank and account in an entity key?                     | Account numbers may repeat across banks; the combination reduces identity collisions.                                                           |
| Why is a threshold a business decision?                            | It controls both missed-risk exposure and investigation workload.                                                                               |



## The Talent Grid | AML Transaction Monitoring Interview Guide

|                                        |                                                                                                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| What is human-in-the-loop?             | Analytics recommends and explains; a trained investigator validates context and decides the final disposition.                                           |
| What would you improve for production? | Add peer groups, time-windowed historical features, feature store, drift monitoring, model/rule versioning, case feedback and stronger privacy controls. |



## Final candidate summary

|                                         |                                                                                                                                    |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>What the project does</b>            | Converts raw transaction data into prioritised AML alerts and account-level risk summaries.                                        |
| <b>What makes it professional</b>       | Uses governed cloud data, data validation, behavioural features, transparent components, measurable thresholds and human review.   |
| <b>Most important interview message</b> | The score is a decision-support mechanism, not proof of laundering.                                                                |
| <b>Best improvement statement</b>       | Production readiness depends on data governance, back-testing, investigator feedback, monitoring and controlled threshold changes. |

**The Talent Grid | Learn the concept, explain the business value, validate the evidence.**